OOP PHP Basics: Class, Object, Property, Method (Beginner Lesson)

Learning outcomes (you will be able to…)

- Explain what **OOP (Object-Oriented Programming)** is and why it's useful in PHP.
- Define a **class** and describe it as a "blueprint" for objects.
- Create **objects** (instances) with `new` and understand "multiple objects from one class".
- Use **properties** to store data, including **public** vs **private** basics.
- Write **methods** (functions inside a class) and use `$this` correctly.
- Use a **constructor** to set up an object.
- Spot common OOP mistakes and debug them quickly.
- Build a small working mini-project using class + objects + properties + methods.

Prerequisites (short)

- Basic PHP syntax: variables, arrays, functions, `if`, loops
- Running PHP 8+ (CLI or local server)

Concept map / ASCII visuals (keep these in mind)

Diagram 1: Class → many Objects

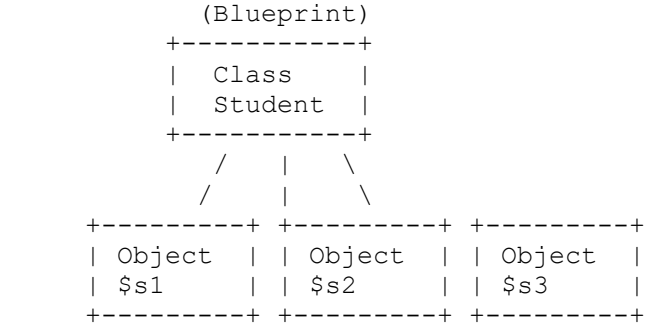
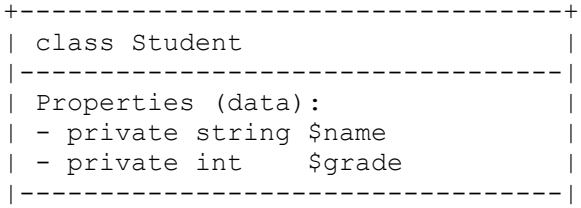


Diagram 2: Inside a class blueprint (properties + methods)



```
| Methods (actions):                |
| + public function getName(): ... |
| + public function promote(): ... |
+-----------------------------------+
```

Diagram 3: Object lifecycle (simple beginner view)

```
new Student(...)  ->  use methods  ->  (optional) unset($student)
      |                    |                  |
  object created       object works       variable removed
```

Diagram 4: Methods change properties (behavior updates data)

```
[Object: $account]
   balance = 100

$account->deposit(50)
        |
        v
   balance = 150
```

Diagram 5: Visibility idea (public vs private)

```
Outside code     Inside class
------------     ------------------------
can call         can access
public methods   private properties safely

Outside: $obj->balance  ✗ (if private)
Outside: $obj->getBalance() ✓
```

Table of contents

A) What is OOP and why it matters (with a simple real-life analogy)

Plain-language explanation

OOP organizes code around "things" (objects) instead of just loose functions and arrays.

Think: a **BankAccount** has:

- data: balance, owner
- actions: deposit, withdraw, show balance

OOP helps because it:

- keeps related code together (data + actions)
- prevents accidental misuse with **visibility** (private/public)
- makes code easier to reuse and extend later

Real-life analogy: "Blueprint and houses"

- A **class** is a *blueprint* (plan).
- An **object** is a *real house built from the blueprint*.
- **Properties** are the house details (color, address).
- **Methods** are what the house can do (lock doors, turn on lights).

Minimal example (smallest working code)

```
<?php
declare(strict_types=1);

// Line 4: Define a class (blueprint)
class Lamp
{
    // Line 7: Property stores data
    public bool $isOn = false;

    // Line 10: Method does an action
    public function turnOn(): void
    {
        // Line 13: Change property value
        $this->isOn = true;
    }
}
```

```
// Line 18: Create an object (real thing)
$lamp = new Lamp();

// Line 21: Use object method
$lamp->turnOn();

// Line 24: Read a property
var_dump($lamp->isOn);
```

Line-by-line (key lines)

- **Line 2** `declare(strict_types=1);` makes PHP stricter about types.
- **Line 4** creates a **class**.
- **Line 18** creates an **object** with `new`.
- **Line 13** uses `$this` to access the current object.

Practical example (realistic scenario)

```
<?php
declare(strict_types=1);

class TodoItem
{
    public string $title = '';
    public bool $isDone = false;

    public function markDone(): void
    {
        $this->isDone = true;
    }
}

$task = new TodoItem();
$task->title = "Finish PHP homework";
$task->markDone();

echo $task->title . " => " . ($task->isDone ? "Done" : "Not done") . PHP_EOL;
```

Weak code (intentionally flawed)

```
<?php
declare(strict_types=1);

class TodoItem
{
    // Weak: everything public, anyone can break rules
    public string $title = '';
    public bool $isDone = false;
}
```

```
$task = new TodoItem();
$task->title = "";
$task->isDone = true; // someone marks done even if title is empty
echo "Saved task!" . PHP_EOL;
```

Why it's weak

- No control over *valid data* (empty title allowed).
- Outside code can change internal state anytime.
- Hard to enforce rules like "title must not be empty".

Improved version (best practice for beginners)

```
<?php
declare(strict_types=1);

class TodoItem
{
    private string $title;
    private bool $isDone = false;

    public function __construct(string $title)
    {
        // Line 11: basic validation
        $trimmed = trim($title);
        if ($trimmed === '') {
            throw new InvalidArgumentException("Title cannot be empty.");
        }
        $this->title = $trimmed;
    }

    public function markDone(): void
    {
        $this->isDone = true;
    }

    public function getTitle(): string
    {
        return $this->title;
    }

    public function isDone(): bool
    {
        return $this->isDone;
    }
}

$task = new TodoItem("Finish PHP homework");
$task->markDone();
```

```
echo $task->getTitle() . " => " . ($task->isDone() ? "Done" : "Not done") .
PHP_EOL;
```

What changed and why

- Properties became **private** so only the class controls them.
- Added a **constructor** to ensure a valid title at creation time.
- Added **getter methods** to safely read values.

Vice versa comparison block

- If you make everything **public** (bad), it causes **uncontrolled changes** and bugs.
- If you keep important data **private** (good), you get **safer objects** and clearer rules.

Student misconceptions (callout #1)

Misconception: "OOP is only for big projects."
Reality: Even small scripts get cleaner when data + actions live together.

Check for understanding (quick questions)

1. In one sentence: what is a **class**?
2. In one sentence: what is an **object**?

B) Class (definition, syntax, rules, naming)

Plain-language explanation

A **class** is a template that defines:

- what data an object stores (**properties**)
- what actions it can perform (**methods**)

Simple rules & naming

- Class names usually use **PascalCase**: `BankAccount`, `StudentProfile`
- One class = one "thing"
- Keep classes focused (don't make a "God class" that does everything)

Minimal example

```
<?php
declare(strict_types=1);

// Line 4: class name in PascalCase
class Greeting
{
    // Line 7: a method inside the class
    public function sayHello(): void
    {
        echo "Hello!" . PHP_EOL;
    }
}

$greeting = new Greeting();
$greeting->sayHello();
```

Line-by-line (key lines)

- **Line 4** defines the **class** name.
- **Line 7** defines a **method** inside the class.
- **Line 14–15** creates and uses an object.

Practical example

```
<?php
declare(strict_types=1);

class TemperatureConverter
{
    public function celsiusToFahrenheit(float $celsius): float
    {
        return ($celsius * 9 / 5) + 32;
    }
}

$converter = new TemperatureConverter();
echo $converter->celsiusToFahrenheit(30) . PHP_EOL; // 86
```

Weak code (intentionally flawed)

```
<?php
declare(strict_types=1);
```

```
class temperature_converter // weak naming style
{
    public function Convert($c) // weak naming + no types
    {
        return ($c * 9 / 5) + 32;
    }
}

$x = new temperature_converter();
echo $x->Convert("30") . PHP_EOL; // string slips in
```

Why it's weak

- Class name style is inconsistent (harder to read).
- Method name is inconsistent and vague.
- No parameter types → unexpected values sneak in.

Improved version

```
<?php
declare(strict_types=1);

class TemperatureConverter
{
    public function celsiusToFahrenheit(float $celsius): float
    {
        return ($celsius * 9 / 5) + 32;
    }
}

$converter = new TemperatureConverter();
echo $converter->celsiusToFahrenheit(30.0) . PHP_EOL;
```

What changed and why

- Used **PascalCase** for class name.
- Used clear method name.
- Added **types** to prevent accidental misuse.

Vice versa comparison block

- If you use messy names + no types (bad), you get **confusing code** and **surprise inputs**.
- If you use clear names + types (good), you get **self-explaining code** and fewer bugs.

Student misconceptions (callout #2)

Misconception: "A class is the same as an array."
Reality: An array holds data; a class defines **data + behavior** together.

Check for understanding

1. What naming style is recommended for class names in PHP?
2. Why are clear class names helpful?

C) Object (creation, multiple objects, instance idea)

Plain-language explanation

An **object** is a real "thing" created from a class.

You can create many objects from one class:

- same blueprint
- different data inside each object

Beginner memory idea:

- Each `new ClassName()` creates a **separate instance**.
- Changing one object's property does **not** change another object.

Minimal example

```
<?php
declare(strict_types=1);

class Counter
{
    public int $value = 0;
}

$a = new Counter();
$b = new Counter();

$a->value = 5;
```

```
echo $a->value . PHP_EOL; // 5
echo $b->value . PHP_EOL; // 0
```

Line-by-line (key lines)

- **Line 9–10** create two different objects.
- **Line 12** changes only `$a`.

Practical example (two users)

```
<?php
declare(strict_types=1);

class User
{
    public string $name = '';
}

$user1 = new User();
$user1->name = "Amina";

$user2 = new User();
$user2->name = "Rafi";

echo $user1->name . PHP_EOL;
echo $user2->name . PHP_EOL;
```

Weak code (intentionally flawed)

```
<?php
declare(strict_types=1);

class User
{
    public string $name = '';
}

$user1 = new User();
$user2 = $user1; // weak: not a new object, just another reference

$user1->name = "Amina";
$user2->name = "Rafi";

echo $user1->name . PHP_EOL; // Rafi (surprise!)
```

Why it’s weak

- `$user2 = $user1;` does **not** create a new object.
- Both variables point to the **same instance**.
- Leads to "Why did my first user change?" confusion.

Improved version

```
<?php
declare(strict_types=1);

class User
{
    public string $name = '';
}

$user1 = new User();
$user1->name = "Amina";

$user2 = new User(); // new instance
$user2->name = "Rafi";

echo $user1->name . PHP_EOL; // Amina
echo $user2->name . PHP_EOL; // Rafi
```

What changed and why

- Created a second object using `new`.
- Each object now has its own separate data.

Vice versa comparison block

- If you assign `$b = $a` (bad), you get **one object with two labels** → unexpected shared changes.
- If you create `$b = new Class()` (good), you get **two independent objects**.

Student misconceptions (callout #3)

Misconception: "`$b = $a` copies the object like arrays."
Reality: It copies the **reference** (two variables pointing to the same object).

Check for understanding

1. What keyword creates an object?

2. If you change `$a->value`, will `$b->value` change automatically?

D) Property (public/private basics, defaults, typed properties optional)

Plain-language explanation

A **property** is a variable inside a class.

- **public**: accessible from outside (`$obj->name`)
- **private**: only accessible inside the class (safer)

Default values:

- `public int $count = 0;`

Typed properties (simple meaning):

- `public int $age;` means age should be an integer.

Minimal example

```
<?php
declare(strict_types=1);

class Profile
{
    public string $displayName = "Guest";
}

$profile = new Profile();
echo $profile->displayName . PHP_EOL;
```

Line-by-line (key lines)

- **Line 6** property with a default value.
- **Line 10** property is read outside because it's **public**.

Practical example (private property + getter)

```
<?php
declare(strict_types=1);

class Product
{
    private string $name;
    private float $price;

    public function __construct(string $name, float $price)
    {
        $this->name = trim($name);
        $this->price = $price;
    }

    public function getPrice(): float
    {
        return $this->price;
    }
}

$product = new Product("Keyboard", 29.99);
echo $product->getPrice() . PHP_EOL;
```

Weak code (intentionally flawed)

```
<?php
declare(strict_types=1);

class Product
{
    public string $name = "";
    public float $price = 0;
}

$product = new Product();
$product->name = "";
$product->price = -500; // weak: invalid price allowed

echo $product->price . PHP_EOL;
```

Why it’s weak

- Allows invalid values (empty name, negative price).
- No “rules” enforced.
- Hard to trust the data later in the program.

Improved version

```
<?php
declare(strict_types=1);

class Product
{
    private string $name;
    private float $price;

    public function __construct(string $name, float $price)
    {
        $trimmed = trim($name);
        if ($trimmed === '') {
            throw new InvalidArgumentException("Product name cannot be
empty.");
        }
        if ($price < 0) {
            throw new InvalidArgumentException("Price cannot be negative.");
        }

        $this->name = $trimmed;
        $this->price = $price;
    }

    public function getName(): string
    {
        return $this->name;
    }

    public function getPrice(): float
    {
        return $this->price;
    }
}

$product = new Product("Keyboard", 29.99);
echo $product->getName() . " costs " . $product->getPrice() . PHP_EOL;
```

What changed and why

- Made properties **private** to protect them.
- Validated input in the **constructor**.
- Exposed safe read access via getters.

Vice versa comparison block

- If properties are **public** without rules (bad), you get **invalid object state**.
- If properties are **private** with validation (good), you get **reliable data**.

Student misconceptions (callout #4)

Misconception: "Private means I can never read it."
Reality: You read it safely using **public methods** like `getPrice()`.

Check for understanding

1. What's the difference between **public** and **private**?
2. Why might we set a default value on a property?

E) Method (parameters, return types, $this, chaining intro optional)

Plain-language explanation

A **method** is a function inside a class.

Methods can:

- take **parameters**
- return values with a **return type**
- use `$this` to access the current object's properties/methods

`$this` means: "this object, the one I'm inside right now."

Minimal example

```
<?php
declare(strict_types=1);

class NameFormatter
{
    public function makeUppercase(string $name): string
    {
        // $this not needed here because we don't use properties
        return strtoupper($name);
    }
}
```

```
$formatter = new NameFormatter();
echo $formatter->makeUppercase("Amina") . PHP_EOL;
```

Line-by-line (key lines)

- **Line 6** method takes `string $name` and returns `string`.
- **Line 13** call the method using `->`.

Practical example (method changes property using $this)

```
<?php
declare(strict_types=1);

class BankAccount
{
    private string $ownerName;
    private float $balance;

    public function __construct(string $ownerName, float $startingBalance)
    {
        $this->ownerName = trim($ownerName);
        $this->balance = $startingBalance;
    }

    public function deposit(float $amount): void
    {
        // Line 17: $this refers to THIS account
        $this->balance = $this->balance + $amount;
    }

    public function getBalance(): float
    {
        return $this->balance;
    }
}

$account = new BankAccount("Rafi", 100.0);
$account->deposit(50.0);
echo $account->getBalance() . PHP_EOL; // 150
```

Line-by-line (key lines)

- **Line 7–11** constructor sets up the object.
- **Line 17** updates a private property via `$this`.

Weak code (intentionally flawed)

```php
<?php
declare(strict_types=1);

class BankAccount
{
    public float $balance = 0;

    public function deposit($amount) // weak: no type
    {
        $this->balance = $this->balance + $amount; // weak: might add string
    }
}

$account = new BankAccount();
$account->balance = -100; // weak: outside code breaks rules
$account->deposit("50abc"); // messy input
echo $account->balance . PHP_EOL;
```

Why it's weak

- `balance` is public → can become negative or nonsense.
- `deposit($amount)` has no type → unsafe input.
- No validation → "money rules" are not enforced.

Improved version (best practice)

```php
<?php
declare(strict_types=1);

class BankAccount
{
    private float $balance;

    public function __construct(float $startingBalance)
    {
        if ($startingBalance < 0) {
            throw new InvalidArgumentException("Starting balance cannot be
negative.");
        }
        $this->balance = $startingBalance;
    }

    public function deposit(float $amount): self
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException("Deposit amount must be
positive.");
        }

        $this->balance += $amount;
```

```
        // Return $this to enable method chaining (optional concept)
        return $this;
    }

    public function withdraw(float $amount): self
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException("Withdraw amount must be
positive.");
        }
        if ($amount > $this->balance) {
            throw new RuntimeException("Insufficient funds.");
        }

        $this->balance -= $amount;
        return $this;
    }

    public function getBalance(): float
    {
        return $this->balance;
    }
}

$account = new BankAccount(100.0);

// Method chaining demo: one object, multiple calls
$account->deposit(50.0)->withdraw(30.0);

echo $account->getBalance() . PHP_EOL; // 120
```

What changed and why

- `balance` is **private** → protected.
- Added type hints (`float`) → safer inputs.
- Added validation rules for deposit/withdraw.
- Returned `self` to allow chaining: `->deposit()->withdraw()`.

Vice versa comparison block

- If methods accept "anything" (bad), you get **weird values** and fragile code.
- If methods use types + validation (good), you get **safe behavior** and predictable results.

Student misconceptions (callout #5)

Misconception: "`$this` is a global variable."
Reality: `$this` exists only **inside class methods** and refers to **the current object**.

Check for understanding

1. What does `$this` refer to?
2. Why might a method return `self`?

F) Mini Project: BankAccount (step-by-step)

We'll build a tiny, real-world class with safe rules:

- Create account with owner + starting balance
- Deposit and withdraw
- Prevent invalid actions

Step 1 — Start with a simple class (minimal working)

```
<?php
declare(strict_types=1);

class BankAccount
{
    public string $ownerName = '';
    public float $balance = 0.0;
}

$account = new BankAccount();
$account->ownerName = "Amina";
$account->balance = 100.0;

echo $account->ownerName . " has " . $account->balance . PHP_EOL;
```

What you should notice

- This works, but it's too open: anyone can set balance to `-9999`.

Check for understanding:
If I write `$account->balance = -500;` does PHP stop me here?

Step 2 — Add a constructor (correct setup)

```
<?php
declare(strict_types=1);

class BankAccount
```

```
{
    public string $ownerName;
    public float $balance;

    public function __construct(string $ownerName, float $startingBalance)
    {
        $this->ownerName = trim($ownerName);
        $this->balance = $startingBalance;
    }
}

$account = new BankAccount("Amina", 100.0);
echo $account->ownerName . " has " . $account->balance . PHP_EOL;
```

Line-by-line highlights

- Constructor runs automatically when you call `new BankAccount(...)`.
- `$this->ownerName` means "this account's ownerName".

Student misconceptions (callout #6)

Misconception: "Constructors are optional so I can ignore them forever."
Reality: Constructors are the cleanest way to ensure objects start in a valid state.

Check for understanding:
What happens if you forget to pass the constructor arguments?

Step 3 — Make properties private + add rules (improved)

```
<?php
declare(strict_types=1);

class BankAccount
{
    private string $ownerName;
    private float $balance;

    public function __construct(string $ownerName, float $startingBalance)
    {
        $trimmed = trim($ownerName);
        if ($trimmed === '') {
            throw new InvalidArgumentException("Owner name cannot be
empty.");
        }
        if ($startingBalance < 0) {
            throw new InvalidArgumentException("Starting balance cannot be
negative.");
        }

        $this->ownerName = $trimmed;
```

```
        $this->balance = $startingBalance;
    }

    public function getOwnerName(): string
    {
        return $this->ownerName;
    }

    public function getBalance(): float
    {
        return $this->balance;
    }
}

$account = new BankAccount("Amina", 100.0);
echo $account->getOwnerName() . " has " . $account->getBalance() . PHP_EOL;
```

What changed and why

- **private** data to protect it.
- Constructor validation prevents invalid accounts.
- Getter methods provide safe access.

Check for understanding:
Why can't we do `$account->balance` anymore?

Step 4 — Add methods that change state (deposit/withdraw)

```
<?php
declare(strict_types=1);

class BankAccount
{
    private string $ownerName;
    private float $balance;

    public function __construct(string $ownerName, float $startingBalance)
    {
        $trimmed = trim($ownerName);
        if ($trimmed === '') {
            throw new InvalidArgumentException("Owner name cannot be
empty.");
        }
        if ($startingBalance < 0) {
            throw new InvalidArgumentException("Starting balance cannot be
negative.");
        }

        $this->ownerName = $trimmed;
        $this->balance = $startingBalance;
    }
```

```
    public function deposit(float $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException("Deposit amount must be
positive.");
        }

        // Line 27: change THIS object's balance
        $this->balance += $amount;
    }

    public function withdraw(float $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException("Withdraw amount must be
positive.");
        }
        if ($amount > $this->balance) {
            throw new RuntimeException("Insufficient funds.");
        }

        $this->balance -= $amount;
    }

    public function getOwnerName(): string
    {
        return $this->ownerName;
    }

    public function getBalance(): float
    {
        return $this->balance;
    }
}

// Demo
$account = new BankAccount("Amina", 100.0);
$account->deposit(50.0);
$account->withdraw(30.0);

echo $account->getOwnerName() . " has " . $account->getBalance() . PHP_EOL;
// 120
```

Weak code version (common beginner approach)

```
<?php
declare(strict_types=1);

class BankAccount
{
    public float $balance = 0.0;

    public function withdraw(float $amount): void
    {
        // weak: allows going negative
```

```
        $this->balance -= $amount;
    }
}

$account = new BankAccount();
$account->balance = 10.0;
$account->withdraw(999.0);
echo $account->balance . PHP_EOL; // -989 (bad)
```

Why it's weak

- No "cannot withdraw more than you have" rule.
- Public `balance` can be edited freely.
- Allows negative balances accidentally.

Improved version (the one above)

- Adds validation and private balance.

Mini project Vice versa comparison block

- If you let outside code set `balance` (bad), you get **broken money logic**.
- If only methods can change `balance` (good), you get **trustworthy account behavior**.

G) Common mistakes + debugging tips

Common mistakes

1. **Forgetting `new`**
   - Wrong: `$account = BankAccount();`
   - Right: `$account = new BankAccount("Amina", 100);`
2. Trying to access **private** properties from outside
   - Wrong: `echo $account->balance;`
   - Right: `echo $account->getBalance();`
3. Not using `$this` inside methods
   - Wrong: `$balance += $amount;`
   - Right: `$this->balance += $amount;`
4. Passing wrong types (especially with strict typing)
   - With `declare(strict_types=1);`, `"100"` is not the same as `100.0`.
5. Creating "fake copies" of objects
   - `$b = $a;` points to the same object.

Debugging tips (practical)

- Use `var_dump($object);` to inspect object structure.
- Print checkpoints: `echo "Reached line X" . PHP_EOL;`
- Catch errors with try/catch during learning:

```
<?php
declare(strict_types=1);

try {
    throw new RuntimeException("Example error");
} catch (Throwable $e) {
    echo $e->getMessage() . PHP_EOL;
}
```

Check for understanding

1. Why does `strict_types` make some code fail faster?
2. What tool can you use to inspect an object quickly?

H) Best practices checklist + code style

Best practices checklist

- ✅ Use **PascalCase** for class names (`BankAccount`)
- ✅ Use clear method names (`deposit`, `withdraw`, `getBalance`)
- ✅ Keep important data **private**
- ✅ Validate inputs in constructor/methods
- ✅ Use type hints (`string`, `int`, `float`) and return types
- ✅ Keep one class focused on one responsibility
- ✅ Prefer methods over direct property edits

Simple code style tips

- One class per file (later, when projects grow)
- Indent consistently (4 spaces is common)
- Don’t abbreviate names: prefer `$ownerName` over `$n`

Vice versa comparison block

- If you skip types + validation (bad), you get **mysterious bugs** later.
- If you add types + simple rules (good), you get **predictable behavior** now.